



COLLEGE CODE : 9100
COLLEGE NAME : Anna University Regional Campus Madurai
NAME : Divya Shri R
DEPARTMENT : Computer Science And Engineering
STUDENT NM ID : B172467ED5BB847A00E4243DEB165CA0
ROLL NO : 910023104014
DATE : 10/25/2025

**COMPLETED PHASE 5 IN THE PROJECT NAMED AS EMPLOYEE
DIRECTORY WITH SEARCH**

SUBMITTED BY,

NAME : Divya Shri R

MOBILE NO : 7708098899

Phase 5: Project Demonstration & Documentation

Title: Employee Directory With Search

1. Final Demo Walkthrough

The **Employee Directory Management System** is a dynamic, web-based application built using **AngularJS**, designed to manage, search, and organize employee details in real-time.

It provides a smooth and interactive user interface where administrators or HR personnel can add, edit, delete, and view employee records efficiently.

Home Screen Overview:

When the application launches, users are presented with a clean, responsive **dashboard** titled “*Employee Directory*”.

The main interface displays a **search bar**, **filter dropdowns**, and a **list of employees** showing essential details like name, role, department, and location.

Each employee record is shown in a **card-style layout**, which makes browsing through multiple profiles easy and visually organized.

Search & Filtering System:

The highlight of the project is its **real-time search feature**.

As users type into the search bar, AngularJS’s two-way data binding instantly filters the displayed employees based on **name, department, or location**.

The system is also equipped with **filter dropdowns** allowing sorting by department or newest/oldest entries.

This makes it ideal for HR personnel managing large employee datasets.

Add & Edit Employee Details:

When the user clicks the “**Add Employee**” button, a modal form opens, allowing them to input:

- First Name
- Last Name
- Role / Designation
- Department
- Contact Details
- Profile Photo URL

On submission, AngularJS immediately updates the displayed list through \$scope without page reloads.

Existing employees can also be edited — clicking the “**Edit**” button populates the same form with existing data, allowing quick modifications.

Delete Employee:

Each employee entry includes a “**Delete**” button.

When clicked, a confirmation popup appears to prevent accidental deletions.

Once confirmed, the entry is removed both from the front-end list and the backend JSON / API data (if connected).

API & Data Integration:

The project supports both **local JSON files** and **backend APIs** for data management.

Using AngularJS \$http service:

- GET requests retrieve the employee list.
- POST adds new employees.
- PUT updates existing ones.
- DELETE removes records.

If the API is unavailable, the app gracefully switches to **local data mode**, ensuring uninterrupted functionality.

User Interface (UI) Design:

The UI is built with **Bootstrap 5**, ensuring a clean, responsive layout across devices.

Color-coded indicators and consistent spacing make data easy to read.

A “glass-style” search bar and animated buttons provide a modern feel.

The interface automatically adapts to desktops, tablets, and mobile devices using a fluid grid layout.

Performance Enhancements:

To improve performance, the app uses:

- **ng-repeat track by** for optimized DOM rendering.
- Lightweight data-binding and scoped watchers.
- Instant updates via AngularJS digest cycle.
- Minimal re-rendering for smooth scrolling and responsiveness.

Even with hundreds of records, the application remains fast and responsive.

Security & Validation:

Input fields are validated using AngularJS's built-in form validation:

- Required fields can't be left empty.
- Email and contact fields use proper regex checks.
- \$sanitize is used to prevent HTML/script injection.

Final Output:

The final deployed app delivers a **seamless, interactive, and professional** employee management experience.

Users can search, sort, add, edit, or delete employees without refreshing the page.

The interface reflects every update instantly, creating an efficient HR tool suitable for small to medium organizations.

2. Screenshots

```
<> index.html > <html> > <head> > <style>
1  <!doctype html>
2  <html lang="en" ng-app="employeeApp">
3  <head>
4  <meta charset="utf-8">
5  <title>Employee Directory - Search Based View</title>
6  <meta name="viewport" content="width=device-width, initial-scale=1">
7
8  <!-- Bootstrap CSS -->
9  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
10
11 <!-- AngularJS -->
12 <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
13
14 <style>
15   body {
16     background: linear-gradient(135deg, #eef3ff, #fdfdfd);
17     font-family: "Poppins", sans-serif;
18     min-height: 100vh;
19   }
20
21   .navbar {
22     background-color: #0d6efd;
23     color: white;
24   }
25
26   .navbar-brand {
27     color: white !important;
28     font-weight: bold;
29   }
30
31   .search-bar {
32     border-radius: 2rem;
33     padding: 0.7rem 1.2rem;
34     box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
```

```

35     font-size: 1.1rem;
36   }
37
38   .card {
39     border-radius: 1rem;
40     box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
41     transition: transform 0.3s;
42   }
43
44   .card:hover {
45     transform: translateY(-5px);
46   }
47
48   .profile-img {
49     border-radius: 50%;
50     margin-bottom: 15px;
51     box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
52   }
53
54   .no-results {
55     text-align: center;
56     color: gray;
57     font-size: 1.2rem;
58     margin-top: 40px;
59   }
60 </style>
61 </head>
62
63 <body ng-controller="employeeCtrl">
```

```

65 <!-- Header -->
66 <nav class="navbar navbar-expand-lg">
67   <div class="container">
68     <a class="navbar-brand" href="#">Employee Directory</a>
69   </div>
70 </nav>
71
72 <div class="container mt-5 text-center">
73
74   <!-- Search Bar -->
75   <div class="d-flex justify-content-center mb-4">
76     <input type="text" class="form-control w-75 search-bar" ng-model="searchText" placeholder="🔍 Search by name, role, or department">
77   </div>
78
79   <!-- Employee Cards (Shown only when searching) -->
80   <div class="row justify-content-center g-4" ng-if="searchText">
81     <div class="col-md-4" ng-repeat="emp in employees | filter:searchText as results">
82       <div class="card text-center p-3">
83         
84         <h5 class="fw-bold">{{emp.name}}</h5>
85         <p class="mb-1"><strong>Age:</strong> {{emp.age}}</p>
86         <p class="mb-1"><strong>Role:</strong> {{emp.role}}</p>
87         <p><strong>Department:</strong> {{emp.department}}</p>
88       </div>
89     </div>
90
91   <!-- No matching results -->
92   <div ng-if="results.length === 0" class="no-results">
93     No matching employees found.
94   </div>
95 </div>

```

```
97     <!-- Message when no search -->
98     <div ng-if="!searchText" class="no-results">
99         | Type in the search box to view employee details.
100     </div>
101
102 </div>
103
104 <!-- Bootstrap JS -->
105 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
106
107 <!-- AngularJS Script -->
108 <script>
109     var app = angular.module('employeeApp', []);
110
111     app.controller('employeeCtrl', function($scope) {
112         $scope.employees = [
113             { id: 1, name: 'John Doe', age: 29, role: 'Software Engineer', department: 'IT', photo: 'https://i.pravatar.cc' },
114             { id: 2, name: 'Jane Smith', age: 34, role: 'HR Manager', department: 'Human Resources', photo: 'https://i.pra' },
115             { id: 3, name: 'Michael Brown', age: 41, role: 'Sales Executive', department: 'Sales', photo: 'https://i.prava' },
116             { id: 4, name: 'Priya Kumar', age: 27, role: 'UI Designer', department: 'Design', photo: 'https://i.pravatar.c' },
117             { id: 5, name: 'Arun Sharma', age: 31, role: 'System Analyst', department: 'IT', photo: 'https://i.pravatar.cc' },
118             { id: 6, name: 'Kavya Patel', age: 26, role: 'Marketing Associate', department: 'Marketing', photo: 'https://i' },
119         ];
120     });
121 </script>
122
123 </body>
124 </html>
```

Output:

Employee Directory

 Search by name, role, or department

Type in the search box to view employee details.

sof



John Doe

Age: 29

Role: Software Engineer

Department: IT

s



John Doe

Age: 29

Role: Software Engineer

Department: IT



Jane Smith

Age: 34

Role: HR Manager

Department: Human Resources



Michael Brown

Age: 41

Role: Sales Executive

Department: Sales

3. Challenges and Solutions

Data Filtering & Search Accuracy:

Challenge:

Initially, the employee search feature was matching only exact strings (e.g., typing “john” wouldn’t match “John Doe”). Also, partial searches like “eng” for “Engineer” weren’t returning results.

Solution:

Implemented AngularJS’s built-in filter directive with two-way binding using ng-model. This automatically performs **case-insensitive, partial matching** across all fields (name, department, role), making search flexible and user-friendly.

Display Control – Hiding Employee Details Initially:

Challenge:

When the page loaded, all employee details were visible by default. The requirement was to show **no employee data initially**, and display details **only when the user searches**.

Solution:

Used AngularJS conditional rendering with ng-if="searchText". This ensures that employee cards are shown **only when the user types something** in the search bar. If the search box is empty, a friendly message prompts the user to search.

Responsive User Interface Design:

Challenge:

Employee cards were not aligned properly on small screens, and text overflowed when long names or department titles were used.

Solution:

Integrated **Bootstrap 5 grid system** and **responsive classes** like col-md-4 and text-center. Adjusted spacing and added soft shadows to make cards visually clean and uniform across desktop, tablet, and mobile devices.

Smooth and Minimalistic UI Design:

Challenge:

The initial user interface looked basic, with flat borders and no modern feel. It didn't meet professional dashboard standards.

Solution:

Enhanced the UI with **CSS gradients**, **rounded corners**, and **box-shadows** to give a modern look. Used a **blue accent theme** for consistency and readability. Added hover transitions to employee cards for subtle interactivity.

Image Handling & Placeholder Issue:

Challenge:

When some employee images failed to load (due to missing URLs), the layout broke, causing uneven card heights.

Solution:

Used ng-src instead of normal src to ensure AngularJS compiles image URLs properly. Also, added fallback avatars using an online avatar API (<https://i.pravatar.cc/>) to ensure every card displays a consistent profile image.

Performance Optimization:

Challenge:

When scaling the application to a large number of employee records, filtering performance decreased due to frequent Angular digest cycles.

Solution:

Optimized the filter usage by limiting it to essential attributes and introduced **lazy rendering** using AngularJS's track-by clause in ng-repeat for efficient DOM updates.

Error Handling & No Result Feedback:

Challenge:

When users typed an invalid search (e.g., “xyz”), the page appeared empty, making users think it was broken.

Solution:

Added a clear feedback message —
"No matching employees found."

This message appears dynamically using ng-if when the filtered list length is zero, ensuring a smooth and informative user experience.

Cross-Browser Compatibility:

Challenge:

Some CSS styles and alignment behaved differently in Chrome, Firefox, and Edge.

Solution:

Used standardized **Bootstrap classes** instead of raw CSS positioning. Verified compatibility across major browsers, ensuring consistent UI appearance and behavior.

Maintainability and Scalability:

Challenge:

As the application grew, maintaining hardcoded employee data within the same script file became inefficient.

Solution:

Refactored the code so that employee data could later be loaded from an external JSON file or API endpoint. This separation improves maintainability and allows easy integration with a backend (like Node.js or Firebase) in future versions.

User Experience & Accessibility:

Challenge:

Visually, the interface worked well, but accessibility (for keyboard or screen readers) was lacking.

Solution:

Improved accessibility by:

- Adding placeholders and labels for search inputs.
- Maintaining contrast ratio for text and background.
- Ensuring focus outline is visible when using the keyboard.

VERSION CONTROL(GITHUB):

<https://github.com/Divyaraji2005/naan-mudhalavn/blob/e8439791f7c7b683805882025204e5d18e0bac77/Employee.html>

TEAM MEMBERS:

1. Divya Shri R
2. Eswara Prathyuksha K S
3. Sowndharya S